

GESp五级直通车

一篇梳理GESp编程能力等级考试五级C++语言全部知识体系，吃透难点，摆平易错点，掌握高频考点。

GESp五级考纲知识点

知识点	考察频率
数据结构：单链表、双链表、循环链表	
算法：	
高精度加法、减法、乘法、除法	
递归与分治	
贪心	
二分查找/二分答案	
复杂度的估算	
数学：初等数论	
--辗转相除法	
--筛法：埃式筛和线性筛	
--唯一分解定理	

一、链表

单链表

双链表

循环链表

二、算法部分

高精度加减乘除

以下是C++实现的高精度加减乘除模板（支持大整数运算），包含详细注释和使用示例：

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class BigInteger {
5  private:
6      vector<int> digits; // 存储各位数字（逆序存储，方便运算）
7      bool isNegative;    // 符号位
8
9      // 去除前导零
10     void removeLeadingZeros() {
11         while (digits.size() > 1 && digits.back() == 0) {
12             digits.pop_back();
13         }
14         if (digits.size() == 1 && digits[0] == 0) {
15             isNegative = false; // 0没有负号
16         }
17     }
18
19     // 比较绝对值大小
20     int compareAbs(const BigInteger& other) const {
21         if (digits.size() != other.digits.size()) {
22             return digits.size() < other.digits.size() ? -1 : 1;
23         }
24         for (int i = digits.size() - 1; i >= 0; --i) {
25             if (digits[i] != other.digits[i]) {
26                 return digits[i] < other.digits[i] ? -1 : 1;
27             }
28         }
29         return 0;
30     }
31
32 public:
33     // 构造函数
34     BigInteger() : isNegative(false) { digits.push_back(0); }
35     BigInteger(const string& s) {
36         int start = 0;
37         if (s[0] == '-') {
38             isNegative = true;
39             start = 1;
```

```

40         } else {
41             isNegative = false;
42         }
43         for (int i = s.size() - 1; i >= start; --i) {
44             digits.push_back(s[i] - '0');
45         }
46         removeLeadingZeros();
47     }
48     BigInteger(long long num) {
49         if (num < 0) {
50             isNegative = true;
51             num = -num;
52         } else {
53             isNegative = false;
54         }
55         do {
56             digits.push_back(num % 10);
57             num /= 10;
58         } while (num);
59     }
60
61     // 输出
62     friend ostream& operator<<(ostream& os, const BigInteger& num) {
63         if (num.isNegative) os << '-';
64         for (int i = num.digits.size() - 1; i >= 0; --i) {
65             os << num.digits[i];
66         }
67         return os;
68     }
69
70     // 加法
71     BigInteger operator+(const BigInteger& other) const {
72         if (isNegative != other.isNegative) {
73             BigInteger tmp = other;
74             tmp.isNegative = !tmp.isNegative;
75             return *this - tmp;
76         }
77
78         BigInteger res;
79         res.isNegative = isNegative;
80         res.digits.clear();
81
82         int carry = 0;
83         int maxLen = max(digits.size(), other.digits.size());
84         for (int i = 0; i < maxLen || carry; ++i) {
85             int sum = carry;
86             if (i < digits.size()) sum += digits[i];

```

```

87         if (i < other.digits.size()) sum += other.digits[i];
88         res.digits.push_back(sum % 10);
89         carry = sum / 10;
90     }
91
92     return res;
93 }
94
95 // 减法
96 BigInteger operator-(const BigInteger& other) const {
97     if (isNegative != other.isNegative) {
98         BigInteger tmp = other;
99         tmp.isNegative = !tmp.isNegative;
100        return *this + tmp;
101    }
102
103    if (compareAbs(other) < 0) {
104        BigInteger res = other - *this;
105        res.isNegative = !isNegative;
106        return res;
107    }
108
109    BigInteger res;
110    res.isNegative = isNegative;
111    res.digits.clear();
112
113    int borrow = 0;
114    for (int i = 0; i < digits.size(); ++i) {
115        int diff = digits[i] - borrow;
116        if (i < other.digits.size()) diff -= other.digits[i];
117        if (diff < 0) {
118            diff += 10;
119            borrow = 1;
120        } else {
121            borrow = 0;
122        }
123        res.digits.push_back(diff);
124    }
125
126    res.removeLeadingZeros();
127    return res;
128 }
129
130 // 乘法
131 BigInteger operator*(const BigInteger& other) const {
132     BigInteger res;
133     res.isNegative = isNegative ^ other.isNegative;

```

```

134         res.digits.resize(digits.size() + other.digits.size(), 0);
135
136         for (int i = 0; i < digits.size(); ++i) {
137             int carry = 0;
138             for (int j = 0; j < other.digits.size() || carry; ++j) {
139                 long long product = res.digits[i + j] +
140                     1LL * digits[i] * (j < other.digits.size()
141 ? other.digits[j] : 0) +
142                     carry;
143                 res.digits[i + j] = product % 10;
144                 carry = product / 10;
145             }
146
147             res.removeLeadingZeros();
148             return res;
149         }
150
151         // 除法 (整除)
152         BigInteger operator/(const BigInteger& other) const {
153             if (other == BigInteger(0)) {
154                 throw runtime_error("Division by zero");
155             }
156
157             BigInteger dividend = *this;
158             BigInteger divisor = other;
159             dividend.isNegative = false;
160             divisor.isNegative = false;
161
162             if (dividend < divisor) {
163                 return BigInteger(0);
164             }
165
166             BigInteger res, current;
167             res.isNegative = isNegative ^ other.isNegative;
168             res.digits.resize(digits.size(), 0);
169
170             for (int i = digits.size() - 1; i >= 0; --i) {
171                 current.digits.insert(current.digits.begin(), dividend.digits[i]);
172                 current.removeLeadingZeros();
173
174                 int quotient = 0;
175                 while (current >= divisor) {
176                     current = current - divisor;
177                     ++quotient;
178                 }
179                 res.digits[i] = quotient;

```

```

180         }
181
182         res.removeLeadingZeros();
183         return res;
184     }
185
186     // 取模
187     BigInteger operator%(const BigInteger& other) const {
188         return *this - (*this / other) * other;
189     }
190
191     // 比较运算符
192     bool operator<(const BigInteger& other) const {
193         if (isNegative != other.isNegative) {
194             return isNegative;
195         }
196         int cmp = compareAbs(other);
197         return isNegative ? cmp > 0 : cmp < 0;
198     }
199     bool operator>(const BigInteger& other) const { return other < *this; }
200     bool operator<=(const BigInteger& other) const { return !(*this > other); }
201     bool operator>=(const BigInteger& other) const { return !(*this < other); }
202     bool operator==(const BigInteger& other) const {
203         return isNegative == other.isNegative && digits == other.digits;
204     }
205     bool operator!=(const BigInteger& other) const { return !(*this == other); }
206 };
207
208 // 使用示例
209 int main() {
210     BigInteger a("12345678901234567890");
211     BigInteger b("98765432109876543210");
212
213     cout << "a = " << a << endl;
214     cout << "b = " << b << endl;
215
216     // 加法
217     cout << "a + b = " << a + b << endl;
218
219     // 减法
220     cout << "b - a = " << b - a << endl;
221
222     // 乘法
223     cout << "a * b = " << a * b << endl;

```

```
224
225     // 除法
226     BigInteger c("123456789");
227     BigInteger d("987");
228     cout << "c / d = " << c / d << endl;
229
230     // 取模
231     cout << "c % d = " << c % d << endl;
232
233     return 0;
234 }
```

关键实现说明

1. 存储方式:

- 使用 `vector<int>` 逆序存储数字（个位在 `digits[0]`）
- 例如：数字 12345 存储为 `[5, 4, 3, 2, 1]`

2. 核心算法:

- 加法：逐位相加处理进位
- 减法：借位处理（确保大数减小数）
- 乘法：模拟竖式乘法（双重循环）
- 除法：模拟竖式除法（试商法）

3. 特殊处理:

- 符号位单独处理
- 去除前导零
- 除零异常检测

4. 时间复杂度:

- 加减法： $O(n)$
- 乘法： $O(n^2)$ （朴素算法）
- 除法： $O(n^2)$ （试商法）

优化建议

1. 乘法优化:

```
1 // 使用Karatsuba算法优化乘法 ( $O(n^{1.585})$ )
2 BigInteger karatsuba(const BigInteger& a, const BigInteger& b) {
3     // 实现略
4 }
```

2. 除法优化:

```
1 // 使用牛顿迭代法优化除法
2 BigInteger newtonDivision(const BigInteger& a, const BigInteger& b) {
3     // 实现略
4 }
```

3. 输入输出优化：

```
1 // 重载>>运算符支持直接输入
2 friend istream& operator>>(istream& is, BigInteger& num) {
3     string s;
4     is >> s;
5     num = BigInteger(s);
6     return is;
7 }
```

示例输出

```
1 a = 12345678901234567890
2 b = 98765432109876543210
3 a + b = 111111111011111111100
4 b - a = 86419753208641975320
5 a * b = 1219326311370217952237463801111263526900
6 c / d = 125083
7 c % d = 78
```

这个模板支持大整数的基本运算，可根据需要进一步扩展功能（如幂运算、位运算等）。

二分查找/二分答案

以下是二分查找上下边界的三种模板及六个实现，包含详细注释和使用示例：

模板分类

1. 基础二分查找（精确匹配）
2. 寻找左边界（第一个 $\geq$ target的位置）
3. 寻找右边界（最后一个 $\leq$ target的位置）

每种模板提供两种写法（左闭右闭区间和左闭右开区间）

1. 基础二分查找（精确匹配）

适用场景：数组中存在且仅存在一个目标值

写法一：左闭右闭区间 [left, right]

```
1 int binarySearch(vector<int>& nums, int target) {
2     int left = 0, right = nums.size() - 1;
3     while (left <= right) { // 注意等号
4         int mid = left + (right - left) / 2;
5         if (nums[mid] == target) {
6             return mid;
7         } else if (nums[mid] < target) {
8             left = mid + 1;
9         } else {
10            right = mid - 1;
11        }
12    }
13    return -1; // 未找到
14 }
```

写法二：左闭右开区间 [left, right)

```
1 int binarySearch(vector<int>& nums, int target) {
2     int left = 0, right = nums.size(); // 右开区间
3     while (left < right) { // 无等号
4         int mid = left + (right - left) / 2;
5         if (nums[mid] == target) {
6             return mid;
7         } else if (nums[mid] < target) {
8             left = mid + 1;
9         } else {
10            right = mid; // 保持右开
11        }
12    }
13    return -1;
14 }
```

2. 寻找左边界（第一个 $\geq$ target的位置）

适用场景：可能有重复元素，找插入位置

写法一：左闭右闭区间

```
1 int leftBound(vector<int>& nums, int target) {
2     int left = 0, right = nums.size() - 1;
3     while (left <= right) {
```

```

4         int mid = left + (right - left) / 2;
5         if (nums[mid] >= target) {
6             right = mid - 1; // 收缩右边界
7         } else {
8             left = mid + 1;
9         }
10    }
11    // 检查越界和是否找到
12    return (left < nums.size() && nums[left] == target) ? left : -1;
13    // 如果仅需插入位置，直接返回left
14 }

```

写法二：左闭右开区间

```

1 int leftBound(vector<int>& nums, int target) {
2     int left = 0, right = nums.size();
3     while (left < right) {
4         int mid = left + (right - left) / 2;
5         if (nums[mid] >= target) {
6             right = mid; // 保持右开
7         } else {
8             left = mid + 1;
9         }
10    }
11    // left ∈ [0, nums.size())
12    return (left < nums.size() && nums[left] == target) ? left : -1;
13 }

```

3. 寻找右边界（最后一个 $\leq$ target的位置）

适用场景：可能有重复元素，找值域上限

写法一：左闭右闭区间

```

1 int rightBound(vector<int>& nums, int target) {
2     int left = 0, right = nums.size() - 1;
3     while (left <= right) {
4         int mid = left + (right - left) / 2;
5         if (nums[mid] <= target) {
6             left = mid + 1; // 收缩左边界
7         } else {
8             right = mid - 1;
9         }
10    }
11    // 检查是否找到
12    return (right >= 0 && nums[right] == target) ? right : -1;
13    // 如果仅需上界位置，直接返回right

```

写法二：左闭右开区间

```
1 int rightBound(vector<int>& nums, int target) {
2     int left = 0, right = nums.size();
3     while (left < right) {
4         int mid = left + (right - left) / 2;
5         if (nums[mid] <= target) {
6             left = mid + 1;
7         } else {
8             right = mid;
9         }
10    }
11    // right ∈ [0, nums.size())
12    return (right > 0 && nums[right - 1] == target) ? right - 1 : -1;
13 }
```

统一记忆技巧

模板类型	关键条件	更新策略	返回值处理
精确查找	nums[mid] == target	直接返回	返回mid或-1
左边界	nums[mid] >= target	收缩右边界	检查left是否有效
右边界	nums[mid] <= target	收缩左边界	检查right-1是否有效

使用示例

```
1 vector<int> nums = {1,2,2,2,3,4};
2
3 cout << binarySearch(nums, 2); // 输出任意一个2的位置（如2）
4 cout << leftBound(nums, 2); // 输出1（第一个2的位置）
5 cout << rightBound(nums, 2); // 输出3（最后一个2的位置）
6 cout << leftBound(nums, 5); // 输出-1（未找到）
7 cout << leftBound(nums, 0); // 输出0（插入位置）
```

常见问题解答

- 1. 为什么左边界模板用 $\geq$ 而右边界用 $\leq$?
 - 左边界找的是第一个不小于target的值
 - 右边界找的是最后一个不大于target的值
- 2. 如何选择区间写法?
 - 左闭右闭：更符合直觉，结束时 $left=right+1$
 - 左闭右开：与STL风格一致，结束时 $left=right$
- 3. 如何处理未找到的情况?
 - 精确查找：直接返回-1
 - 边界查找：返回的 $left/right$ 可表示插入位置
- 4. 死循环如何避免?
 - 确保每次循环区间都在缩小
 - 检查mid计算是否可能溢出（使用 $left + (right-left)/2$ ）

算法复杂度

常见时间复杂度总结

以下是计算机科学中最常见的时间复杂度类别，按效率从高到低排列：

时间复杂度	名称	描述	典型算法
$O(1)$	常数时间	执行时间与输入规模无关	访问数组元素、哈希表查找、栈/队列操作
$O(\log n)$	对数时间	执行时间随输入规模呈对数增长	二分查找、平衡二叉搜索树操作、堆操作
$O(\sqrt{n})$	平方根时间	执行时间与输入规模的平方根成正比	某些质数判断算法、跳表查找
$O(n)$	线性时间	执行时间与输入规模成正比	遍历数组/链表、线性搜索、计数排序

时间复杂度	名称	描述	典型算法
$O(n \log n)$	线性对数时间	执行时间与输入规模的对数和规模的乘积成正比	高效排序算法（快排、归并、堆排序）、FFT
$O(n^2)$	平方时间	执行时间与输入规模的平方成正比	冒泡/选择/插入排序、矩阵乘法（朴素）、嵌套循环
$O(n^3)$	立方时间	执行时间与输入规模的立方成正比	矩阵乘法（朴素）、Floyd-Warshall算法、三维嵌套循环
$O(2^n)$	指数时间	执行时间随输入规模呈指数增长	暴力破解、子集生成、旅行商问题（穷举）
$O(n!)$	阶乘时间	执行时间随输入规模呈阶乘增长	排列生成、旅行商问题（穷举）、图同构检测（最坏情况）

详细说明及示例

1. $O(1)$ - 常数时间

- 最佳时间复杂度
- 示例：

```
1 | int getFirst(int arr[]) {
2 |     return arr[0];    // 直接访问数组第一个元素
3 | }
```

2. $O(\log n)$ - 对数时间

- 高效搜索算法
- 示例（二分查找）：

```
1 | int binarySearch(int arr[], int n, int target) {
2 |     int left = 0, right = n - 1;
3 |     while (left <= right) {
4 |         int mid = left + (right - left) / 2;
5 |         if (arr[mid] == target) return mid;
6 |         if (arr[mid] < target) left = mid + 1;
7 |         else right = mid - 1;
8 |     }
9 |     return -1;
10 | }
```

3. $O(n)$ - 线性时间

- 单层循环操作
- 示例：

```
1 int sumArray(int arr[], int n) {
2     int sum = 0;
3     for (int i = 0; i < n; i++) {
4         sum += arr[i]; // 遍历整个数组
5     }
6     return sum;
7 }
```

4. $O(n \log n)$ - 线性对数时间

- 高效排序算法
- 示例（归并排序）：

```
1 void mergeSort(int arr[], int l, int r) {
2     if (l < r) {
3         int m = l + (r - l) / 2;
4         mergeSort(arr, l, m); //  $O(\log n)$  递归深度
5         mergeSort(arr, m + 1, r);
6         merge(arr, l, m, r); //  $O(n)$  合并操作
7     }
8 }
```

5. $O(n^2)$ - 平方时间

- 双重嵌套循环
- 示例（冒泡排序）：

```
1 void bubbleSort(int arr[], int n) {
2     for (int i = 0; i < n - 1; i++) {
3         for (int j = 0; j < n - i - 1; j++) {
4             if (arr[j] > arr[j + 1]) {
5                 swap(arr[j], arr[j + 1]);
6             }
7         }
8     }
9 }
```

6. $O(2^n)$ - 指数时间

- 递归问题（无优化）
- 示例（斐波那契数列）：

```

1  int fibonacci(int n) {
2      if (n <= 1) return n;
3      return fibonacci(n - 1) + fibonacci(n - 2);  // 指数级递归调用
4  }

```

时间复杂度曲线图

```

1  时间消耗
2  ^
3  | O(n!)      *****
4  | O(2^n)     *****
5  | O(n^3)     *****
6  | O(n^2)     *****
7  | O(n log n) *****
8  | O(n)       ***
9  | O(log n)   *
10 | O(1)       .
11 +-----> 输入规模 (n)

```

实际应用中的时间复杂度选择

1. 小规模数据 ($n < 100$)

- 任何算法都可接受
- 常选择简单实现的 $O(n^2)$ 算法

2. 中等规模数据 ($100 < n < 10,000$)

- 优先选择 $O(n \log n)$ 算法
- 避免 $O(n^3)$ 和更高复杂度

3. 大规模数据 ($n > 10,000$)

- 必须使用 $O(n)$ 或 $O(n \log n)$ 算法
- 对性能关键部分考虑 $O(\log n)$ 算法
- 绝对避免指数级算法

4. 超大规模数据 ($n > 10^6$)

- 精心优化 $O(n)$ 算法
- 考虑并行计算和分布式处理
- 使用近似算法替代精确解法

重要注意事项

1. 大O表示法的含义

- 描述算法的最坏情况增长率
- 忽略常数因子和低阶项
- 例如： $O(2n^2 + 3n + 4) = O(n^2)$

2. 空间复杂度

- 类似概念，描述内存使用随输入规模的增长
- 常见： $O(1)$ 、 $O(n)$ 、 $O(n^2)$ 等

3. 均摊分析

- 某些操作可能偶尔耗时，但整体高效
- 示例：动态数组的插入操作（多数 $O(1)$ ，扩容时 $O(n)$ ）

4. 实际性能因素

- 常数因子： $O(n)$ 可能比 $O(\log n)$ 快（当 n 小时）
- 硬件特性：缓存命中、分支预测
- 输入数据特性：部分算法在特定输入下表现更好

理解这些时间复杂度类别有助于在设计和选择算法时做出明智决策，平衡性能和实现复杂度。

三、初等数论部分：

辗转相除法（欧几里得算法）详解

1. 算法概括

用途：计算两个非负整数的最大公约数（GCD）

核心思想： $\gcd(a,b) = \gcd(b, a \bmod b)$

时间复杂度： $O(\log \min(a,b))$

优势：比穷举法高效，适合大数运算

2. 数学证明

定理： $\gcd(a,b) = \gcd(b, a \bmod b)$

证明：

1. 设 $d = \gcd(a,b)$ ，则 $d|a$ 且 $d|b$
2. 设 $a = kb + r$ （其中 $r = a \bmod b$ ）
3. 因为 $d|a$ 且 $d|kb$ ，故 $d|(a - kb) \Rightarrow d|r$
4. 因此 d 也是 b 和 r 的公约数
5. 反过来可证 $\gcd(b,r)$ 也是 a 和 b 的公约数

6. 故 $\gcd(a,b) = \gcd(b,r)$

3.C++ 代码模板

基础递归版

```
1 int gcd(int a, int b) {  
2     return b == 0 ? a : gcd(b, a % b);  
3 }
```

迭代版 (推荐)

```
1 int gcd(int a, int b) {  
2     while (b != 0) {  
3         int temp = b;  
4         b = a % b;  
5         a = temp;  
6     }  
7     return a;  
8 }
```

处理负数和零

```
1 int gcd(int a, int b) {  
2     a = abs(a);  
3     b = abs(b);  
4     if (a == 0) return b;  
5     if (b == 0) return a;  
6     return gcd(b, a % b);  
7 }
```

6. 相关算法

- 最小公倍数 (LCM) :

```
1 int lcm(int a, int b) {  
2     return a / gcd(a, b) * b;    // 先除后乘防溢出  
3 }
```

- 二进制优化版 (适合大数) :

```
1 int binary_gcd(int a, int b) {
2     if (a == 0) return b;
3     if (b == 0) return a;
4     if ((a & 1) == 0 && (b & 1) == 0)
5         return binary_gcd(a >> 1, b >> 1) << 1;
6     else if ((a & 1) == 0)
7         return binary_gcd(a >> 1, b);
8     else if ((b & 1) == 0)
9         return binary_gcd(a, b >> 1);
10    else
11        return binary_gcd(abs(a - b), min(a, b));
12 }
```

复杂度对比

方法	时间复杂度	空间复杂度
辗转相除法	$O(\log n)$	$O(1)$
穷举法	$O(n)$	$O(1)$
二进制优化版	$O(\log n)$	$O(1)$

注: $n = \min(a,b)$

筛法

筛法求素数详解

一、埃拉托斯特尼筛法（埃式筛）

算法思想：

- 1. 从2开始，将所有素数的倍数标记为非素数
- 2. 时间复杂度: $O(n \log \log n)$

数学证明：

- 每个合数都会被其最小质因数筛掉
- 对于素数p，从 p^2 开始标记（因为 $p \times k$ ($k < p$) 已被更小的素数筛过）

C++模板：

```

1  vector<bool> sieve(int n) {
2      vector<bool> is_prime(n+1, true);
3      is_prime[0] = is_prime[1] = false;
4      for (int i = 2; i * i <= n; ++i) {
5          if (is_prime[i]) {
6              for (int j = i * i; j <= n; j += i) {
7                  is_prime[j] = false;
8              }
9          }
10     }
11     return is_prime;
12 }

```

优化版本:

```

1  vector<int> get_primes(int n) {
2      vector<bool> is_prime(n+1, true);
3      vector<int> primes;
4      is_prime[0] = is_prime[1] = false;
5      for (int i = 2; i <= n; ++i) {
6          if (is_prime[i]) {
7              primes.push_back(i);
8              if ((long long)i * i <= n) {
9                  for (int j = i * i; j <= n; j += i) {
10                     is_prime[j] = false;
11                 }
12             }
13         }
14     }
15     return primes;
16 }

```

二、欧拉筛（线性筛）

算法思想:

1. 每个合数只被其最小质因数筛掉
2. 时间复杂度: $O(n)$

数学证明:

- 设合数 $N = p \times M$ (p 是 N 的最小质因数)
- 当 i 枚举到 M 时, $p \times M$ 会被筛掉
- 保证每个合数只被筛一次

C++模板:

```
1 vector<int> linear_sieve(int n) {
2     vector<int> primes;
3     vector<bool> is_prime(n+1, true);
4     is_prime[0] = is_prime[1] = false;
5     for (int i = 2; i <= n; ++i) {
6         if (is_prime[i]) {
7             primes.push_back(i);
8         }
9         for (int p : primes) {
10             if (i * p > n) break;
11             is_prime[i * p] = false;
12             if (i % p == 0) break; // 关键：保证只被最小质因数筛
13         }
14     }
15     return primes;
16 }
```

带最小质因数记录的优化版：

```
1 vector<int> min_prime_factors(int n) {
2     vector<int> spf(n+1, 0);
3     vector<int> primes;
4     for (int i = 2; i <= n; ++i) {
5         if (spf[i] == 0) {
6             spf[i] = i;
7             primes.push_back(i);
8         }
9         for (int p : primes) {
10             if (p > spf[i] || i * p > n) break;
11             spf[i * p] = p;
12         }
13     }
14     return spf; // spf[x] 存储x的最小质因数
15 }
```

三、对比分析

特性	埃式筛	线性筛
时间复杂度	$O(n \log \log n)$	$O(n)$
空间复杂度	$O(n)$	$O(n)$
筛除次数	多次	每个合数仅一次

特性	埃式筛	线性筛
适用场景	简单场景	需要高效预处理
额外功能	仅素数标记	可记录最小质因数

四、使用示例

```
1  int main() {
2      // 埃式筛使用
3      auto primes_eratosthenes = get_primes(100);
4      cout << "埃式筛素数个数: " << primes_eratosthenes.size() << endl;
5
6      // 线性筛使用
7      auto primes_euler = linear_sieve(100);
8      cout << "线性筛素数个数: " << primes_euler.size() << endl;
9
10     // 最小质因数筛
11     auto spf = min_prime_factors(100);
12     cout << "36的最小质因数: " << spf[36] << endl;
13
14     return 0;
15 }
```

五、算法应用

- 1. 素数判定：直接查询筛表
- 2. 质因数分解：利用最小质因数数组快速分解
- 3. 欧拉函数计算：结合素数表高效计算 $\varphi(n)$
- 4. 区间筛法：处理大范围素数问题

区间筛法示例（筛[a,b]范围内的素数）：

```
1  vector<bool> segment_sieve(long long a, long long b) {
2      vector<bool> is_prime(b - a + 1, true);
3      if (a == 1) is_prime[0] = false;
4      for (long long p = 2; p * p <= b; ++p) {
5          for (long long j = max(p * p, (a + p - 1) / p * p); j <= b; j += p) {
6              is_prime[j - a] = false;
7          }
8      }
9      return is_prime;
10 }
```

唯一分解定理

唯一分解定理（算术基本定理）

1. 定理陈述

任何大于1的正整数都可以唯一地表示为素数的乘积（不考虑因子顺序）。形式化表示为：

对于任意整数 $n > 1$ ，存在唯一分解：

$$n = p_1^{\alpha_1} \times p_2^{\alpha_2} \times \cdots \times p_k^{\alpha_k}$$

其中：

- $p_1 < p_2 < \cdots < p_k$ 是素数
- α_i 是正整数

2. 数学证明

存在性证明（归纳法）：

1. 基础： $n = 2$ 是素数，成立
2. 假设所有 $m < n$ 可分解
3. 若 n 是素数，自身即分解
4. 若 n 是合数，则 $n = a \times b$ ($1 < a, b < n$)
5. 由归纳假设， a 和 b 可分解，故 n 可分解

唯一性证明（反证法）：

1. 假设存在两种不同分解：
$$n = p_1^{\alpha_1} \cdots p_k^{\alpha_k} = q_1^{\beta_1} \cdots q_m^{\beta_m}$$
2. 由素数的定义，必有 p_1 整除某个 q_j
3. 因为 q_j 是素数，故 $p_1 = q_j$
4. 两边约去 p_1 后继续推导，最终所有因子对应相等

3. C++ 实现模板

质因数分解函数

```
1 vector<pair<int, int>> prime_factorize(int n) {
2     vector<pair<int, int>> factors; // 存储 (素数, 指数)
3
4     // 处理2的因子
5     if (n % 2 == 0) {
6         int cnt = 0;
7         while (n % 2 == 0) {
8             n /= 2;
9             cnt++;
10        }
```

```

10     }
11     factors.emplace_back(2, cnt);
12 }
13
14 // 处理奇数因子
15 for (int i = 3; i * i <= n; i += 2) {
16     if (n % i == 0) {
17         int cnt = 0;
18         while (n % i == 0) {
19             n /= i;
20             cnt++;
21         }
22         factors.emplace_back(i, cnt);
23     }
24 }
25
26 // 处理剩余的素数
27 if (n > 1) {
28     factors.emplace_back(n, 1);
29 }
30
31 return factors;
32 }

```

预处理最小质因数优化版

```

1 vector<pair<int, int>> fast_factorize(int n, const vector<int>& spf) {
2     vector<pair<int, int>> factors;
3     while (n > 1) {
4         int p = spf[n];
5         int cnt = 0;
6         while (n % p == 0) {
7             n /= p;
8             cnt++;
9         }
10        factors.emplace_back(p, cnt);
11    }
12    return factors;
13 }
14
15 // 需要先通过线性筛预处理spf数组
16 // vector<int> spf = min_prime_factors(MAXN);

```

4. 应用场景

1. 计算约数个数:

$$d(n) = (\alpha_1 + 1)(\alpha_2 + 1) \cdots (\alpha_k + 1)$$

2. 计算约数和:

$$\sigma(n) = \prod_{i=1}^k \frac{p_i^{\alpha_i+1} - 1}{p_i - 1}$$

3. GCD/LCM计算:

- GCD: 取各素数的最小指数
- LCM: 取各素数的最大指数

4. 同余方程求解: 中国剩余定理基础

5. 完整使用示例

```
1  int main() {
2      int n = 360;
3      auto factors = prime_factorize(n);
4
5      cout << n << " = ";
6      for (int i = 0; i < factors.size(); ++i) {
7          if (i > 0) cout << " * ";
8          cout << factors[i].first << "^" << factors[i].second;
9      }
10     cout << endl;
11
12     // 计算约数个数
13     int divisors = 1;
14     for (auto [p, exp] : factors) {
15         divisors *= (exp + 1);
16     }
17     cout << "约数个数: " << divisors << endl;
18
19     return 0;
20 }
```

输出:

```
1  360 = 2^3 * 3^2 * 5^1
2  约数个数: 24
```


6. 算法分析

方法	时间复杂度	空间复杂度	适用场景
试除法	$O(\sqrt{n})$	$O(1)$	单次分解小数字
预处理最小质因数法	$O(\log n)$	$O(n)$	需要多次分解大数字
Pollard's Rho	$O(n^{1/4})$	$O(1)$	超大数分解